JAVA SDE-2 INTERVIEW PREPARATION SERIES

FRAMEWORKS, DATA, AND MESSAGING - BOOK 06 OF 12 - STUDY STEP FW-06

Spring Data for Java Backend Engineers

Repository Contracts, Query Patterns, Transactions, and Store Boundaries

BY

Vinay Reddy Kalluri

A pragmatic, interview-oriented guide for repository boundaries, query patterns, transactions, pagination, MongoDB, Redis, observability, and production-safe Spring Data usage.

SPRING DATA | REPOSITORIES | TRANSACTIONS | PAGINATION | MULTI-STORES

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `FW 05 - Spring Boot and REST`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Build strong repository-first reasoning, query-shape confidence, transaction correctness, and performance awareness before and during SDE-2 interviews.

Readiness map

Decision	Evidence
START HERE	A query method causes wrong ordering without explicit deterministic sort; A repository call hides pagination, lock, or isolation behavior; A write-path bug appears only under concurrency or duplicate keys; An interview answer must distinguish repository abstraction from store truth.
PREPARE FIRST	FW-01 MySQL; FW-02 Hibernate and JPA; FW-03 Spring Framework; FW-04 Spring Boot; Java 21 and basic SQL or document-store literacy.
MOVE ON WHEN	Design repository contracts from aggregate boundaries and safety requirements; Choose derived, declared, native, or custom query strategies correctly; Control pagination, sorting, locking, flush, and testing semantics under load; Integrate MongoDB and Redis boundaries without confusing persistence roles; Explain repository decisions with performance, failure, and observability evidence.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Frameworks Segment Position

FW 05 - Spring Boot and REST	**YOU ARE HERE** **FW 06 - Spring Data**	FW 07 - MongoDB

A note from Vinay

Repository convenience can hide query count, flush timing, pagination drift, and transaction scope. Keep one eye on the abstraction and the other on the SQL and connection underneath it.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Start Here - Your Route Through This Volume 2

Part I - Core Learning 5

Chapter 1 - Learning Path and Persistence First Principles . . . 5

Chapter 2 - What Spring Data Is and When Not To Use It . . . 8

Chapter 3 - Domain Models, Aggregate Boundaries, and Repository Interfaces . . . 12

Chapter 4 - Repository APIs and Honest CRUD Contracts . . . 14

Chapter 5 - Derived Query Methods and Caveats . . . 18

Chapter 6 - Declared Queries, Native SQL, and Projections . . . 21

Chapter 7 - Pagination, Sorting, Streaming, and Window Patterns . . . 25

Chapter 8 - Transactions, Flush Boundaries, and Write Strategy . . . 29

Chapter 9 - Fetch Plans, the N+1 Pattern, and Performance Badges . . . 32

Chapter 10 - Locking and Concurrency Protection . . . 35

Chapter 11 - Custom Queries and Specification Patterns . . . 39

Chapter 12 - Auditing, Soft Delete, and Entity Lifecycle . . . 42

Chapter 13 - Spring Data MongoDB Foundations . . . 45

Chapter 14 - Spring Data Redis for Cache and Rate-State . . . 48

Chapter 15 - Testing Data Paths, Observability, and Regression Harness . . . 51

Chapter 16 - Common Interview Traps for Spring Data . . . 54

CONTENTS NAVIGATION

Chapter 17 - Practice, Solutions, Readiness, and Sources 56

Chapter 18 - Java 21 Spring Data Readiness Companion 62

Part II - Practice and Handoff **74**

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Learning Path and Persistence First Principles

Spring Data is useful when you already understand one thing: **all persistence has an explicit contract**. Repositories are only the last mile of that contract.

> **A note from Vinay:** When repository code looks like one harmless line, pause and ask what crosses the database boundary. That habit is more valuable in an interview than remembering another annotation. We will begin with SQL and `EntityManager`, then add Spring Data only after the hidden work is visible.

Repository code is convenient for CRUD and simple filters, but repository methods are not the storage system. The interview objective is to preserve database truth even when repository abstractions feel easier.

Why this volume exists

- You already know Java fundamentals and Spring Core.
- You know how transactional code behaves in principle.
- You need a clear path from repository methods to SQL/MongoDB/Redis behavior.

Learning path map

TEXT

```
Domain model -> Aggregate boundaries -> Repository contract -> Query strategy ->
Transaction + flush + isolation decisions -> Pagination + sorting + locks ->
Template fallback when abstraction leaks -> Testing with real stores -> Trade-off discussion
```

The runtime path to draw before answering

```text
HTTP command or scheduled job
          |
          v
application service  -- owns business invariant and transaction
          |
          v
repository proxy     -- derives or selects query implementation
          |
          v
store module         -- JPA EntityManager / MongoTemplate / Redis connection
          |
          v
driver + connection + database/broker-visible operation
          |
          v
flush / acknowledgement / commit / rollback
```

The proxy is not the database. A successful Java method call may only have changed a managed entity; SQL can be delayed until flush. A returned page may require a second count query. A lock annotation still depends on the transaction and database engine. At every arrow, name the request, SQL or native command, resource held, failure, and durable state.

The three-level implementation rule

High-value examples in this volume use three levels:

1. **Native contract:** SQL/JDBC or store-native operation states exactly what must happen.
2. **Lower-level Spring adapter:** `EntityManager`, `JdbcTemplate`, `MongoTemplate`, or `RedisTemplate` keeps the operation explicit.
3. **Repository abstraction:** Spring Data removes repeated adapter code when its contract remains honest.

This is not an invitation to reimplement a database or Spring Data. It is a method for seeing what the abstraction delegates.

What this book is and is not

This book teaches:

- Correctly defining repository contracts.
- Reading repository behavior as a policy layer, not storage truth.
- Where query derivation helps and where it hides cost.
- Debugging interview-ready snippets for correctness and performance.

This is not a deep dive into internals of query parser/optimizer, Spring internals, or complete admin operations.

WHY IT WORKS | Core invariant for interviews

In every interview explanation, separate three layers:

1. **Method contract** - Java method names, arguments, return type, exception profile.
2. **Behavior contract** - SQL/NoSQL query, filters, sort, and projection.
3. **Failure contract** - what happens on duplicate keys, missing rows, lock timeout, stale data.

If you can state these three layers before writing code, you are interview-ready for Spring Data discussions.

Quick check

1. Why can repository code feel too simple during interviews?
2. Which behavior is always database behavior, not repository behavior?
3. What should be in a repository boundary before adding method names?

Practice

- **Foundation:** Identify the aggregate boundaries in a simple `Order` service and write a three-method repository contract.
- **Foundation:** Map repository method return type decisions for `find` vs `get` variants.
- **Interview Core:** Give a non-technical explanation of repository convenience vs storage guarantees.
- **SDE-2 Follow-up:** Explain how repository choice changes rollback reasoning, monitoring, and performance.

TRY IT | Readiness checkpoint

Move to Master Chapter 1 when you can answer: "If a repository method returns a `List`, what are the behavior questions not answered by that method signature?".

SDE-2 CORE CHAPTER

Chapter 2 - What Spring Data Is and When Not To Use It

Spring Data gives you repository patterns plus query abstraction. It does not replace relational design, transaction semantics, or indexing.

Core idea

Think of Spring Data as a **convenience adapter**:

- A repository method is still backed by SQL, Mongo queries, or Redis operations.
- You must still choose transaction boundaries and consistency trade-offs.
- You must still read execution behavior under load.

Spring Data Commons supplies shared repository concepts. A store module such as Spring Data JPA, MongoDB, or Redis implements only the contracts that make sense for that store. Similar Java method names do **not** make SQL rows, MongoDB documents, and Redis values share transaction, query, or durability semantics.

See the abstraction being added

Without a repository, a focused JPA adapter is ordinary Java:

JAVA

```java
final class JpaOrderLookup {
    private final EntityManager entityManager;

    JpaOrderLookup(EntityManager entityManager) {
        this.entityManager = entityManager;
    }

    Optional<OrderEntity> find(long id) {
        return Optional.ofNullable(entityManager.find(OrderEntity.class, id));
    }
}
```

Spring Data can generate the common adapter:

JAVA

```java
interface OrderRepository extends Repository<OrderEntity, Long> {
    Optional<OrderEntity> findById(Long id);
}
```

Both eventually use the persistence provider and driver. The repository saves boilerplate; it does not change entity lifecycle, SQL isolation, indexes, or commit semantics.

Decision first

Use a repository when:

- Access is CRUD-heavy and bounded.
- Domain operations are simple and stable.
- Query language can stay close to model language.

Use a lower-level `JdbcTemplate`, `EntityManager`, Mongo `MongoTemplate`, or Redis API when:

- You need non-standard joins, windowed updates, bulk writes, or vendor-specific operators.
- Performance reasoning is dominated by generated query shape.
- You need deterministic lock and retry behavior for each step.

Also choose a custom repository fragment when most queries are simple but one use case needs lower-level control. The decision is per operation, not one framework choice for the entire application.

Failure and edge matrix

Situation	Hidden risk	Safer decision
method returns thousands of entities	heap growth and long persistence context	projection plus a bounded `Slice`, cursor, or stream lifecycle
query name contains several `Or`/`And` parts	parser precedence hides business grouping	declared query or `Specification` with tests
write calls a remote provider inside transaction	locks and connections remain held; outcome can be unknown	short local transaction plus durable intent/outbox
ORM-generated SQL is the bottleneck	Java method hides joins and row multiplication	inspect SQL/plan, then projection, custom query, or JDBC
multiple stores are updated	no repository creates cross-store atomicity	explicit workflow, idempotency, reconciliation, or saga

Interview-sized model

TEXT

```
Method contract + module style + domain semantics
                 |
                 v
            Repository method
                 |
           store query + plan
                 |
      row/document/cache side effect
```

Common myth corrected

- **Myth:** "Repository methods are enough for all query quality."

  Reality: Repository methods are only a mapping point. Cost, locks, and visibility still come from the store.

- **Myth:** "If repository returns a `List`, all rows are deterministic."

  Reality: Sorting and tie-breakers are your responsibility unless explicitly specified.

Debugging exercise

1. Repository method `findByStatus(String)` runs fast for one developer dataset.
2. In production, API latency spikes with status cardinality change.
3. What questions do you ask before adding more derived methods?

Expected investigation:

- query shape and index usage,
- sort/order determinism,
- pagination or cursor strategy,
- transaction and cache interactions.

Quick check

1. What does repository abstraction remove from your code?
2. What does it not remove?
3. Why is ordering an explicit API concern?

Practice

- **Foundation:** Choose repository vs template for one simple read API.
- **Interview Core:** Explain an example where derived methods would be harmful.
- **SDE-2 Follow-up:** State 3 reasons to keep a custom query for read-heavy endpoints.

Interviewer question and model answer

Interviewer: If Spring Data removes repository implementations, why would an SDE-2 engineer ever use `EntityManager` or `JdbcTemplate`?

Model answer: Spring Data removes recurring adapter code, not the need to control a query. I keep repositories for bounded CRUD and clear predicates. I use a custom fragment, `EntityManager`, or JDBC when the use case needs vendor SQL, bulk mutation, window functions, deterministic locking, or a projection whose cost must stay visible. I choose per operation, keep the transaction in the application service, and prove the choice with generated SQL, the target database plan, and integration tests.

SDE-2 CORE CHAPTER

Chapter 3 - Domain Models, Aggregate Boundaries, and Repository Interfaces

Before writing repository methods, define what your aggregate owns and what it guarantees. A repository should protect the aggregate root contract, not leak transport details.

Core model

A domain aggregate in an interview-safe model has:

- **Identity:** immutable or validated key type.
- **Invariant checks:** invariants enforced in behavior.
- **Boundary methods:** explicit state transitions.
- **Persistence intent:** what may be read, written, and in which transaction scope.

WHY IT WORKS | Repository shape that supports invariants

Prefer focused methods:

- `save(Order order)`
- `findByOrderIdWithLines(Long orderId)` (if needed by boundary)
- `existsByReferenceNumber(String referenceNumber)`

Avoid exposing broad internals with generic methods:

- `findAll()` with no filters,
- `updateBy...` bypassing aggregate behavior,
- `saveAll` without validation.

Example with behavior contract

JAVA

```java
interface OrderRepository {
    Order requireByOrderId(UUID id);     // application-owned port contract
    Optional<Order> findByOrderId(UUID id);
    boolean isReferenceNumberTaken(String referenceNumber);
    void reserveForAccount(UUID accountId, UUID productId, int quantity);
}
```

`requireByOrderId` is an application-owned port name; its adapter translates absence to a domain-specific exception. Do not assume the word `get` in a Spring Data derived query universally means "throw when missing." Return type, nullability metadata, query cardinality, and the specific API define missing-result behavior.

`reserveForAccount` is a domain command, not a data dump method.

Why this helps interviews

Interviewers often ask:

- How do you prevent partial state changes?
- Why not expose only one generic save method?
- How does persistence stay aligned to invariants?

These are answered by showing domain boundaries, not just SQL knowledge.

Quick check

1. What is an aggregate boundary in plain terms?
2. Why is a command-oriented repository method safer than a generic update helper?
3. How do repository contracts prevent repeated missing validation?

Debugging exercise

Given `OrderRepository.save(Order)` and `OrderService.markShipped(Order)`:

- Find the issue if `markShipped` directly toggles status in multiple places.
- Add one rule that makes this method resilient to duplicate transitions.

Expected answer: centralize transition checks before state write and keep repository methods narrow.

Practice

- **Foundation:** Write three repository method names for one `Product` aggregate.
- **Interview Core:** Classify one method as query, command, and validation boundary.
- **SDE-2 Follow-up:** Explain why a single generic repository method can become a bug hotspot.

SDE-2 CORE CHAPTER

Chapter 4 - Repository APIs and Honest CRUD Contracts

Spring Data repository style removes repeated adapter code. It becomes dangerous when a broad inherited API is mistaken for a domain contract.

Start with the Commons hierarchy

The important interfaces are capabilities, not maturity levels:

Interface	Adds	Interview caution
`Repository<T, ID>`	marker and selective method exposure	use when a narrow contract matters
`CrudRepository<T, ID>`	save, identity lookup, existence, count, delete	methods do not define aggregate policy or endpoint safety
`ListCrudRepository<T, ID>`	list-shaped variants of multi-result CRUD	`findAll` is still unbounded
`PagingAndSortingRepository<T, ID>`	`Pageable` and `Sort` access	ordering and count cost remain explicit decisions
`JpaRepository<T, ID>`	JPA-oriented operations including flush/batch helpers	exposes ORM lifecycle details; do not leak it directly to controllers

Store modules can add or omit behavior. Program to the smallest useful interface, and expose application-specific methods from the service boundary.

The baseline contract

For each repository, interview-level clarity usually starts with these intents:

1. **Fetch by identity** (`findById`, `existsById`, `getBy...`)
2. **Create/update by command** (`save`)
3. **Delete when legal** (`deleteBy...`, `delete`)
4. **List by bounded window** (explicit sort + pagination)

Each intent has a failure mode:

- missing row,
- duplicate identity,
- invalid transition,

- stale snapshot,
- write lost under concurrency.

Missing-result semantics

Do not infer behavior only from `find`, `get`, or `read` in a derived method name.

JAVA

```java
interface OrderRepository extends Repository<OrderEntity, Long> {
    Optional<OrderEntity> findByRequestKey(String requestKey);
    OrderEntity findRequiredByExternalId(String externalId);
}
```

For supported Spring Data modules, an `Optional` communicates normal absence. A non-wrapper single result may be nullable under the module's nullability rules or may trigger an empty-result exception when declared non-null. Multiple matches can raise an incorrect-result-size exception. `JpaRepository.getReferenceById` is different again: it can return a lazy JPA reference whose missing row is discovered only when state is accessed. Verify the exact module/version contract and translate it at the application boundary.

Existence is not counting

Use `existsBy...` to ask whether at least one matching row exists. Use `countBy...` only when the count is the required result.

SQL

```sql
-- existence intent; an engine can stop after the first match
select 1 from customer_order where request_key = ? limit 1;

-- counting intent; all matching rows contribute
select count(*) from customer_order where status = ?;
```

Generated SQL is store/provider specific, so inspect it. Neither operation is an application health check: health should test the service's readiness contract with bounded work, not count a business table.

Behavioral semantics you should name

- The application service must establish aggregate invariants before `save`; the repository does not invent them.
- `find` should define what happens when no result exists.
- `delete` should define soft-delete, hard-delete, and audit behavior.
- List operations should define a stable order and bounded size.

Method naming pattern preview

```text
findBy{field}           -> absence follows the declared return/nullability contract
getBy{field}            -> another query subject; the prefix alone does not promise an exception
getReferenceById        -> JPA reference; access can fail later if the row is missing
deleteBy{field}         -> scope and returned count/result must be deliberate
countBy{field}          -> exact count intent, potentially more work than existence
existsBy{field}         -> existence intent; still requires an index and bounded predicate
```

For SDE-2 discussions, explicit `Optional`/exception behavior is often clearer than implicit null handling.

Common confusion

- `findAll()` without limit is rarely interview-safe for endpoints.
- `delete` by relation fields can silently remove multiple rows if naming is ambiguous.
- `save` on detached objects needs clear merge/attach policy.

Failure and edge matrix

Case	Observable result	Design response
no single result	empty wrapper, null, or exception by contract	normalize at service boundary
multiple rows for a single-result query	incorrect-result-size failure	protect uniqueness in the database
duplicate insert	often discovered at flush/commit	classify the known constraint and roll back
detached entity passed to JPA `save`	provider may merge a copied state graph	prefer load-and-mutate command handling
broad derived delete	many rows and locks	require tenant/identity scope and assert affected count
`findAll` on growing table	unbounded memory, context, and response	bounded projection/page/cursor

Quick check

1. Why is `findAll()` usually a scaling smell?
2. Which method names can accidentally hide an update-by-accident bug?
3. When is returning `Optional` clearer than returning null?

Debugging exercise

A ticket service uses `deleteByCustomerId(String)` and runs at high scale.

- What is the hidden risk?
- How do you correct it with safer method naming and preconditions?

Expected correction: restrict delete scope, assert caller intent, and validate count/range before deletion.

Practice

- **Foundation:** Rewrite three repository methods to add explicit return types and failure behavior.
- **Interview Core:** Choose between `find`, `get`, and `exists` for two use cases.
- **SDE-2 Follow-up:** Explain a production issue caused by an overly broad `deleteBy...` method.

Interviewer question and model answer

Interviewer: What is the difference between `findById`, `getReferenceById`, `existsById`, and `countByStatus`?

Model answer: `findById` performs identity lookup and represents absence with `Optional`. In JPA, `getReferenceById` may return a lazy reference without reading the row immediately, so missing data can fail when the reference is initialized. `existsById` asks a boolean question and may use an existence-shaped query. `countByStatus` must calculate the number of matches and should not be used when I only need existence. I still inspect generated SQL and indexes, and I never expose these methods directly as an unbounded HTTP contract.

SDE-2 CORE CHAPTER

Chapter 5 - Derived Query Methods and Caveats

Derived query methods are fast to write and easy to read. They are not free.

How derived methods are built

Method names are parsed into predicates and ordering clauses.

TEXT

```
findByStatusAndPriorityGreaterThanOrderByCreatedAtDesc
```

That single line expresses status, numeric filter, and descending order.

At repository creation, the store module parses the subject (`find`, `exists`, `count`, `delete`, `First`, `Top`) and predicate. It resolves property paths against the domain type and selects a module-specific query implementation. A misspelled property normally fails application startup rather than waiting for the first request.

JAVA

```
interface OrderRepository extends Repository<OrderEntity, Long> {
    Slice<OrderSummary> findByStatusOrderByCreatedAtDescIdDesc(
            String status, Pageable page);

    boolean existsByTenantIdAndRequestKey(long tenantId, String requestKey);
}
```

`Slice` can answer whether another window exists without promising a total count. The explicit `id` tie-breaker makes equal timestamps deterministic.

Why naming can still fail interviews

People memorize fragments but miss intent and runtime consequences:

- Missing sort is equivalent to nondeterministic order.
- Implicit joins may force large query plans.
- `And` and `Or` follow the query parser's grouping rules, but method names cannot express arbitrary parentheses clearly.
- Parameter naming that changes does not change runtime behavior.

TRACE IT | Mini dry run

Input: `findByStatusAndCreatedAtAfterOrderByCreatedAtDesc("OPEN", cutoff)`

Output behavior:

- Filter on status and timestamp.
- Return newest-open items first.
- Stable only if secondary tie-breakers are explicit (for example `ThenById`).

If an index is missing on `status`, query cost can still be high.

Likely relational shape, subject to provider and dialect:

SQL

```
select ...
from customer_order
where status = ? and created_at > ?
order by created_at desc, id desc
limit ?;
```

The Java name cannot prove that the index supports this filter/order or that the projection is narrow.

Interview guidance

Use derived methods for:

- clear single-aggregate rules,
- high readability,
- low query variance.

Avoid derived methods for:

- complex boolean groups,
- vendor-specific functions,
- heavy joins,
- pagination requiring deterministic cross-field ordering.

Debugging exercise

Repository method: `findByNameContainingOrDescriptionContainingAndPriorityBetweenOrderByCreatedAt(String a, String b, int min, int max)`

Explain the parser grouping and why the business intent is still hard to audit.

Expected: the derived grammar treats the `And` part as belonging to the second `Or` branch, approximately `name contains a OR (description contains b AND priority between min and max)`. If priority must apply to both text fields, a long method name cannot add those parentheses naturally. Use a declared query or a tested `Specification` that expresses `(name OR description) AND priority` explicitly.

Quick check

1. What runtime behavior is hidden behind a long derived name?
2. Why should you name deterministic sort keys explicitly?
3. When should you switch to declared/typed queries?

Practice

- **Foundation:** Convert one vague method name into explicit predicates and sort.
- **Interview Core:** Rewrite a long derived method as two simpler repository methods.
- **SDE-2 Follow-up:** Diagnose whether a specific derived method can use a covering index.

Interviewer question and model answer

Interviewer: Are derived query methods evaluated in Java after loading entities?

Model answer: Normally no. The repository factory parses the method into a store query, and Spring Data JPA asks the provider to execute JPQL/SQL. I confirm the actual statement because joins, case handling, null parameters, pagination, and count queries vary by method and module. If the name hides complex grouping or performance, I switch to an explicit query or custom fragment and test the generated SQL and result order.

SDE-2 CORE CHAPTER

Chapter 6 - Declared Queries, Native SQL, and Projections

Declared queries keep behavior readable while enabling fine control.

Three query channels

1. **Derived methods:** generated from method names.
2. **Declared queries (`@Query`):** explicit, readable, often portable.
3. **Native queries:** full control, vendor-specific power, stronger responsibility.

A fourth channel is a **custom repository fragment** implemented with `EntityManager`, `JdbcTemplate`, or another store template. It is often clearer than forcing a complex use case through annotations.

One query at three levels

Start with the result and database work:

SQL

```sql
select id, status, created_at
from customer_order
where tenant_id = ? and status = ?
order by created_at desc, id desc
limit ?;
```

An explicit JPA adapter keeps the query visible:

JAVA

```java
List<OrderSummary> findLatest(long tenantId, String status, int limit) {
    return entityManager.createQuery("""
            select new example.OrderSummary(o.id, o.status, o.createdAt)
            from OrderEntity o
            where o.tenantId = :tenantId and o.status = :status
            order by o.createdAt desc, o.id desc
            """, OrderSummary.class)
        .setParameter("tenantId", tenantId)
        .setParameter("status", status)
        .setMaxResults(limit)
        .getResultList();
}
```

The repository form removes adapter plumbing:

JAVA

```java
@Query("""
       select new example.OrderSummary(o.id, o.status, o.createdAt)
       from OrderEntity o
       where o.tenantId = :tenantId and o.status = :status
       order by o.createdAt desc, o.id desc
       """)
Slice<OrderSummary> findLatest(
        long tenantId, String status, Pageable page);
```

These are not automatically identical. A `Page` may add a count query; a `Slice` commonly requests enough rows to determine continuation. Parameter binding, SQL generation, null handling, and limit syntax depend on provider and dialect.

Choosing the right channel

Use declared/native query when:

- you need explicit joins,
- you need aggregation or windowed sorting,
- return shape needs fields not present in entity graph,
- optimizer behavior must be controlled.

Use derived when:

- query stays inside method-level intent,
- team clarity is more important than SQL novelty,
- execution remains bounded.

Projection pattern

Projection reduces payload and accidental update risk.

TEXT

```
Entity table (wide)
    |
    +---> projection interface/class
          (selected columns only)
```

If projection includes computed fields, confirm if they are DB-native or application computed.

Projection choices include interface projections, DTO/record constructor projections, and dynamic projections. Closed projections can select a narrow shape; open expression-based projections can require more source state and obscure work. Never serialize a managed entity merely because it already has getters.

Common trap

- Native query changes with each DB version.
- Declared query may still load extra columns unless projection is correct.
- Projections can hide expensive joins behind cheap-looking method names.
- A native paged query may need an explicit correct `countQuery`; a guessed count can disagree with grouping or joins.
- Bulk JPQL/native mutation bypasses normal managed-entity dirty checking and can leave the persistence context stale until it is cleared or refreshed.

Edge and failure matrix

Choice	Failure mode	Evidence
DTO projection	constructor/type mismatch	context or query integration test
interface projection	getter alias mismatch	representative result mapping
native query	dialect/schema drift	target-engine test and migration compatibility
paged join	inflated count or duplicate roots	result/count assertions with one-to-many data
bulk update	managed objects retain old state	clear/refresh test in same transaction
dynamic sort	unsafe property or unsupported expression	allow-list and repository test

Quick check

1. Why does a projection not automatically mean performance is good?
2. Which layer owns SQL portability checks for native query?
3. When should you avoid native query in interviews?

Debugging exercise

Two queries fetch the same fields, one derived and one declared.

Latency jumps 4x in production.

Explain the 4 checks you run first.

Expected checks:

- compare execution plans,
- verify indexes,

- validate sort + filter order,
- confirm projection columns and cardinality.

Practice

- **Foundation:** Write one declared query equivalent to a derived method.
- **Interview Core:** Name why native queries should include test data with vendor versions.
- **SDE-2 Follow-up:** Explain projection risks with entity-to-DTO mapping.

Interviewer question and model answer

Interviewer: When would you choose `@Query`, a `Specification`, `EntityManager`, or JDBC?

Model answer: I use `@Query` for a stable explicit JPQL query, a `Specification` when a bounded set of optional filters must compose, `EntityManager` for JPA behavior that does not fit a repository annotation cleanly, and JDBC for SQL-first reads, bulk work, window functions, or vendor features where ORM mapping adds little. I keep parameters bound, use a narrow projection, make ordering and limits explicit, and verify the generated or written SQL on the target database.

SDE-2 CORE CHAPTER

Chapter 7 - Pagination, Sorting, Streaming, and Window Patterns

Pagination prevents loading everything. It also adds complexity in ordering and cursor correctness.

Base model

For interview-safe pagination, define:

- page size,
- sort fields,
- deterministic tie-breaker,
- no accidental duplicate overlap,
- bounded response contract.

Choose the return contract deliberately

Contract	What it promises	Hidden/extra work
`List<T>` with `Pageable`	one bounded list	no total/continuation metadata unless application adds it
`Slice<T>`	content plus whether another slice exists	commonly fetches enough rows to detect continuation; no total count promise
`Page<T>`	content, page metadata, and total	can execute a separate count query
`Window<T>`	a scroll window and continuation position	requires stable scroll semantics and supported repository method shape
`Stream<T>`	incremental consumption	transaction, connection, and stream must remain open and be closed deterministically

Use a `Page` only when the product needs an exact total. On a large filtered relation, counting can cost more than fetching one result window.

Offset vs cursor

Offset pagination is easier but can be expensive and unstable with deletes/inserts. Cursor pagination is better for deep scrolling and real-time data.

TEXT

```
Offset pagination:
SELECT ... LIMIT 20 OFFSET 40

Cursor pagination:
WHERE (created_at, id) > (:cursorCreatedAt, :cursorId)
ORDER BY created_at, id
LIMIT 20
```

These are separate models:

TEXT

```
offset state = page number or numeric offset
cursor state = last ordered key tuple, for example (createdAt, id)
```

Do not advance both an offset and a cursor. A cursor is bound to normalized filters, sort direction, tenant/authorization scope, and version. If the client changes those inputs, reject or restart the traversal.

For descending `(created_at, id)` order, continuation is:

SQL

```
where created_at < :last_created_at
   or (created_at = :last_created_at and id < :last_id)
order by created_at desc, id desc
limit :limit_plus_one;
```

Mixed ascending/descending fields need a carefully derived lexicographic predicate; do not copy a tuple comparison blindly.

Practical rules

- Always use deterministic secondary sort (for example `createdAt DESC, id DESC`).
- Keep sort keys aligned with supporting indexes.
- Return explicit page metadata (`hasNext`, `nextCursor`) from the service contract.
- Cap page size before constructing `PageRequest`.
- Allow-list externally supplied sort properties; do not expose arbitrary entity paths.
- Treat a cursor as opaque and integrity-protected when tampering changes scope or cost.

Failure matrix

Failure	Cause	Correction
duplicate/missing rows across offset pages	inserts/deletes move positions	keyset cursor or documented snapshot
duplicate/missing cursor rows	non-total or mutable ordering key	unique immutable tie-breaker and explicit live semantics
slow deep page	database still skips many rows	aligned keyset predicate/index
expensive `Page`	count query scans/joins large result	`Slice`, cached/approximate total, or separate product action
connection exhaustion	stream escapes transaction or is not closed	try-with-resources and bounded processing
cross-tenant cursor reuse	cursor does not bind scope	derive authorization server-side and sign/bind cursor

Quick check

1. Why is `ORDER BY created_at DESC` alone often insufficient?
2. Where do offset-based pages become unreliable?
3. How does cursor pagination change resume semantics after deletes?

Debugging exercise

An API uses `findTop20ByOrderByCreatedAtDesc`. At page 2, users sometimes see duplicates after new inserts.

Why this happens and what fix you apply first.

Expected answer: deterministic tie-breakers and cursor strategy aligned with stable order.

Practice

- **Foundation:** Draw offset vs cursor for 12 records and one insert between page reads.
- **Interview Core:** Write a safe page contract for a `GET /orders` endpoint.
- **SDE-2 Follow-up:** Define failure cases for streaming repository calls in high concurrency.

Interviewer question and model answer

Interviewer: Why is cursor pagination not simply `PageRequest.of(page + 1, size)`?

Model answer: `PageRequest` describes offset/page-number navigation. Cursor pagination continues from the last total-order key, such as `(createdAt,id)`, using a range predicate aligned to the sort. I do not combine the two. I bind the cursor to filters and scope, cap the limit, fetch `limit + 1`, and document behavior when ordering keys change. Cursor traversal improves deep-page work and live stability but does not create a database snapshot.

SDE-2 CORE CHAPTER

Chapter 8 - Transactions, Flush Boundaries, and Write Strategy

Many Spring Data interview issues are not repository syntax. They are transaction lifecycle issues.

Transaction boundary model

TEXT

```
Service method entry
    -> begin tx
    -> repository write(s)
    -> flush/dirty checks
    -> commit or rollback
```

If transaction annotations are absent, each repository call may still run, but exception handling and visibility become confusing.

The application-service method normally owns the transaction because it owns the invariant. Repository methods participate; they do not make a multi-step use case atomic merely by being repositories.

Flush behavior (why it matters)

`flush` controls when pending changes are sent to the store.

- At commit: predictable, fewer intermediate SQL operations.
- Manual early flush: detects failures sooner.
- Too many manual flushes can increase round trips.

An explicit early flush can be useful when the application deliberately needs a known database constraint result before continuing local work. After a persistence exception, treat the transaction as failed and roll it back; do not keep using a possibly inconsistent persistence context.

Do **not** use flush to justify a slow remote call inside the database transaction. Flushing sends SQL, but the transaction can still hold locks and a pooled connection until commit or rollback. Commit local state plus a durable intent/outbox, then call the remote system outside the local transaction with idempotency and reconciliation.

Propagation and isolation quick map

- `REQUIRED`: join existing transaction or start one.
- `REQUIRES_NEW`: suspend the caller's transaction and create an independent physical transaction when supported. It is not a retry mechanism and can require another pooled connection.
- Isolation tuning appears only after correctness and lock trade-offs are mapped.

A retry policy is separate: classify a transient database failure, roll back the failed transaction, refresh/re-read state, start a fresh transaction, keep one logical idempotency identity, and stop within an attempt/deadline budget.

Write path from method to durable state

TEXT

```
service proxy enters
  -> transaction manager obtains/binds connection
  -> repository proxy selects implementation
  -> EntityManager changes managed state
  -> flush converts changes to SQL
  -> database checks constraints/locks/version predicate
  -> transaction commits
  -> only now is local state durable
```

SQL can also flush before commit when a query must observe pending changes, when `saveAndFlush`/`flush` is called, or under provider-specific flush rules. The Java line containing `save` is not a universal durability point.

Failure and recovery matrix

Failure	State at observation	Correct response
validation before transaction	no database work	correct request/domain input
unique constraint at flush	transaction failed/rollback required	classify known constraint; return conflict or replay
optimistic version conflict	another commit won	re-read, reapply if business-safe, bounded fresh-transaction retry
deadlock victim/serialization failure	database rolled back transaction	retry whole idempotent local unit if vendor code is classified
remote timeout after local commit	remote outcome unknown; DB committed	status lookup/retry same idempotency key/reconcile
`REQUIRES_NEW` waits for connection	outer transaction may hold one while inner needs another	revisit semantics and bound concurrency/pool demand

Common mistake

- Assuming repository write failure happens at method return.
- Assuming read-only methods can safely call writing repository operations.
- Ignoring optimistic lock exceptions and treating them as fatal framework issues.

Quick check

1. What does transaction boundary ownership belong to in clean architecture?
2. Why does flush not make a following remote call atomic or cheap?
3. Why must a retry begin after the failed transaction has rolled back?

Debugging exercise

Service A calls Service B with propagation defaults and performs a remote payment call.

Payment succeeds, then DB check fails on B.

Explain the rollback behavior and interview-safe fix.

Expected: separate the boundaries intentionally, define compensation strategy where business requires, or move side-effect outside the transaction with a durable intent.

Practice

- **Foundation:** Draw the service-repository-transaction boundary for one command endpoint.
- **Interview Core:** Describe `REQUIRES_NEW` in one sentence, including its connection and atomicity cost.
- **SDE-2 Follow-up:** For your named database, explain its documented lock types and deadlock evidence. Do not assume vendor-independent lock escalation.

Interviewer question and model answer

Interviewer: Should I call `saveAndFlush`, then invoke a payment provider so I know the order was written?

Model answer: Not as a default. Flush can surface a database constraint earlier, but the transaction is still uncommitted and can retain locks and its connection during the remote call. The provider cannot participate in that local atomic commit, and a timeout has an unknown remote outcome. I commit an order state plus outbox or payment intent in one short transaction, call the provider with a stable idempotency key outside it, and update/reconcile status in another transaction. I use `saveAndFlush` only when early SQL synchronization is itself required and tested.

SDE-2 CORE CHAPTER

Chapter 9 - Fetch Plans, the N+1 Pattern, and Performance Badges

If your data model reads a parent with many children, N+1 can hide in any abstraction.

N+1 in simple words

1 query for parent rows + N queries for each parent's child collection.

This is a classic interview failure because method naming may hide this behavior.

Prevention layers

- **Join fetch / fetch join** where result shape is stable.
- **Batch fetch / chunked secondary query** with controlled window.
- **Projection DTO** for endpoint-only payload.

```text
parent query ----> children query per parent (bad)
parent + fetch join --> complete in bounded single query (good for bounded graphs)
```

Decision rule

Choose a fetch plan for the use case when:

- relationship is always needed,
- row explosion is bounded,
- paging strategy prevents huge memory spikes.

Avoid making every association statically eager. One endpoint may need lines while another needs only order summaries. Prefer a bounded fetch join/entity graph, controlled batching, or explicit projection/query by relationship IDs according to result cardinality.

SQL consequences

```text
20 orders + lazy lines accessed in a loop
  -> 1 order SELECT + up to 20 line SELECTs

20 orders + one collection fetch join
  -> 1 SQL statement, but one row per order-line pair

20 order summaries projected from order table
  -> 1 narrow SQL statement, no managed child graph
```

A collection fetch join can create a Cartesian product when several to-many associations are joined, and pagination over a collection fetch may be applied in memory or rejected depending on provider/version. Inspect SQL, rows, statements, heap, and response shape-not only query count.

Edge matrix

Strategy	Useful when	Main edge
entity graph/fetch join	one bounded graph is required	row multiplication and paging
batch fetching	several lazy associations are needed	extra round trips and batch tuning
DTO projection	read endpoint needs a stable narrow shape	explicit mapping and duplicate/group logic
two-step IDs then graph	page roots plus to-many data	preserve root order and bound ID list

Quick check

1. How does projection reduce impact of fetch joins?
2. Why is N+1 still possible with repository methods?
3. What is a safe first diagnostic query for suspected N+1?

Debugging exercise

Endpoint returns one order page with nested items. QA reports 250 SQL statements for 20 rows.

What do you do in order?

Expected:

- inspect query log,
- check repository method and transaction boundary,
- add graph strategy,
- add assertion test for query count.

Practice

- **Foundation:** Explain N+1 for `Order` and `OrderLine` in one paragraph.
- **Interview Core:** Propose one fix without changing API shape.
- **SDE-2 Follow-up:** Compare fetch join vs batch fetch for high-cardinality relationships.

Interviewer question and model answer

Interviewer: Should I fix N+1 by marking every association eager?

Model answer: No. Static eager loading shifts the surprise to queries that do not need the relationship and can create row or object-graph explosions. I identify the endpoint's result shape, capture SQL and query count, then choose a bounded entity graph/fetch join, batch fetch, or DTO projection. For a pageable root with a to-many relationship, I often page root IDs first and fetch the bounded graph second, then test order, statements, rows, and memory.

SDE-2 CORE CHAPTER

Chapter 10 - Locking and Concurrency Protection

Optimistic and pessimistic locking are interview-level decisions. A repository method without lock policy is often incomplete for SDE-2 readiness.

Optimistic lock baseline

Optimistic locking uses version fields and checks updates.

- Detects stale write by version mismatch.
- Low contention path, good throughput.
- Requires exception handling and retry policy in service layer.

In JPA the common mechanism is a version column:

JAVA

```java
@Entity
class OrderEntity {
    @Id
    private Long id;

    @Version
    private long version;
}
```

An update is conceptually guarded by the old version:

SQL

```sql
update customer_order
set status = ?, version = version + 1
where id = ? and version = ?;
```

Zero affected rows means the expected snapshot did not win. Reload and retry only when the business transition remains valid; never loop blindly around `save` with the same stale state.

Pessimistic lock baseline

Pessimistic locking blocks writers/ readers at row level.

- Prevents conflicts in contention hotspots.
- Increases wait/lock timeout risk.
- Needs clear timeout and fallback.

JAVA

```java
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select o from OrderEntity o where o.id = :id")
Optional<OrderEntity> findForUpdateById(long id);
```

The call needs an active transaction. SQL syntax, lock scope, gap/predicate behavior, timeout exceptions, and deadlock detection are database- and dialect-specific.

Interview-safe locking flow

TEXT

```text
Read with version / lock intent
      |
      v
Apply invariants and mutate state
      |
      v
Save and handle stale write / timeout exceptions
```

Vendor boundary: do not claim universal lock escalation

Lock escalation is a documented behavior of some engines, not a portable JPA rule. MySQL/InnoDB, PostgreSQL, SQL Server, and MongoDB expose different row, key, range, predicate, document, and transaction models. Name the target store, exact statement and index, isolation/concern, rows or ranges touched, and evidence such as a deadlock graph or lock-wait view. Spring's `@Lock` selects a JPA lock mode; it does not normalize those internals.

Failure matrix

Symptom	Likely class	Next evidence
optimistic exception	version predicate affected zero rows	competing command and current version
lock timeout	incompatible lock held beyond wait policy	blocker transaction, query, index, hold time
deadlock victim	cyclic wait chosen for rollback	engine deadlock graph and acquisition order
high conflicts after retry	hot aggregate or stale workflow	contention rate and transition design
pool wait during locked work	too much concurrent or long transaction work	pool pending, transaction duration, useful DB concurrency

Debugging exercise

Two workers edit the same row using optimistic locking.

- Worker 1 reads version 3 and saves to version 4.
- Worker 2 reads version 3 and saves.

What happens and what does worker 2 need to do?

Expected: worker 2 gets stale-write signal and retries from refreshed state.

Quick check

1. Why can optimistic locking fit a low-conflict path?
2. When might pessimistic locking be justified?
3. What makes lock timeout strategy interview-safe?

Practice

- **Foundation:** Write a pseudo retry loop for stale writes.
- **Interview Core:** Explain why lock scope should be minimal.
- **SDE-2 Follow-up:** Compare lock behavior across MySQL and MongoDB at a high level.

Interviewer question and model answer

Interviewer: When would you choose optimistic versus pessimistic locking?

Model answer: I start from the invariant and measured contention. Optimistic versioning is a good default when conflicts are uncommon and a command can be re-read and safely retried. Pessimistic locking can fit a short, high-contention critical section when bounded waiting is preferable to repeated conflicts. I keep either transaction short, use an index-supported predicate, set a bounded wait, classify vendor exceptions, and never claim JPA makes lock behavior identical across databases.

SDE-2 CORE CHAPTER

Chapter 11 - Custom Queries and Specification Patterns

At some point derived methods become brittle. Specification-style composition helps when predicates change per request.

Why this exists

Feature flags and filter dashboards create combinational query logic. Repeating derived names for every combination does not scale.

Pattern model

TEXT

```
Input filters -> predicate composer -> validated query -> repository execution -> response page
```

Use this when:

- filter combinations exceed a few variants,
- business rules cross aggregate boundaries,
- sorting and paging must remain stable across dynamic conditions.

Construction advice

- Build pure predicate builders.
- Keep each rule independently testable.
- Always preserve deterministic ordering for pageable results.

Spring Data JPA's `JpaSpecificationExecutor` composes JPA Criteria predicates:

JAVA

```java
static Specification<OrderEntity> hasStatus(String status) {
    return (root, query, criteria) ->
            status == null ? criteria.conjunction()
                    : criteria.equal(root.get("status"), status);
}

Specification<OrderEntity> filter = hasStatus(status)
        .and(createdAtOrAfter(from))
        .and(tenantIs(tenantId));
```

Always add tenant/authorization scope server-side. Do not let a nullable user filter remove a mandatory security predicate. Dynamic predicates still become SQL; optional `OR` patterns and functions can make an index unusable.

Minimal pseudo flow

- `status == OPEN`
- `amount between minAmount and maxAmount`
- `createdAt >= from` and `< to`
- `priority in ('HIGH','URGENT')`

Result: one composed query path rather than 24 method variants.

Common mistake

- Building one huge specification that re-evaluates conditions in the wrong order and breaks index use.
- Passing unvalidated request values into query predicates.
- Accepting arbitrary property names for sort/path traversal.
- Reusing one mutable predicate builder across requests.

Edge matrix

Input	Required behavior
no optional filters	bounded tenant-scoped query, never global `findAll`
invalid range	reject before query construction
empty collection filter	define "no matches" versus "ignore filter" explicitly
unknown sort field	reject through allow-list
large `IN` list	cap, stage, or redesign; do not generate unbounded SQL

Quick check

1. Why do derived methods become hard to maintain with dynamic filters?
2. How do you keep specification code readable?
3. What test proves predicate composition is safe?

Debugging exercise

A search endpoint accepts optional `status`, `priority`, and date range and has wrong results with null combinations.

Identify the first three validation checks.

Expected: null-safe condition builder, validated ranges, explicit sort defaults + tests for boundary nulls.

Practice

- **Foundation:** Draft three optional filters and map them to composable predicates.
- **Interview Core:** Explain how composition avoids method explosion.
- **SDE-2 Follow-up:** Show where you validate user input before query building.

Interviewer question and model answer

Interviewer: Are Specifications automatically efficient because the database optimizes them?

Model answer: No. A Specification only composes Criteria predicates. The resulting SQL can still contain broad `OR`, functions on indexed columns, large `IN` lists, joins, and an expensive count query. I validate and cap filters, always add tenant scope, fix the sort to a total order, inspect SQL/plans on representative data, and use an explicit read model or JDBC when the dynamic shape stops being predictable.

SDE-2 CORE CHAPTER

Chapter 12 - Auditing, Soft Delete, and Entity Lifecycle

Repository code often appears in interview problems that need immutable history, compliance fields, and reversible deletes.

Core concepts

Auditing

Audit fields (`createdBy`, `createdAt`, `updatedBy`, `updatedAt`) are operational evidence.

Spring Data auditing can populate these fields through `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`, and an `AuditorAware` provider. That is convenient metadata population, not a tamper-proof audit log. Bulk SQL, another application, or a broken principal provider can bypass or misattribute it.

Soft delete

Soft delete keeps row history and allows replaying compliance checks.

Soft delete is a product/legal choice, not a universal requirement. It does not by itself satisfy retention, restore, privacy erasure, or immutable audit requirements.

Lifecycle visibility

Repository methods should expose lifecycle state explicitly:

- created/active,
- archived,
- deleted/expired,
- restored.

Interview pattern

Use dedicated repository methods for lifecycle states and avoid reusing broad `findAll` over mixed states.

TEXT

```
read model
  -> active-only methods
  -> explicit archive/read-only methods
  -> explicit restore/reject methods
```

Common issue

Soft-delete flags without index support can cause full scans. Not including tenant/user separation in indexes can create accidental cross-tenant exposure.

Uniqueness also needs an explicit policy: may a deleted username or external ID be reused? The answer may require a partial/generated index, a lifecycle column in the key, anonymization, or permanent reservation depending on the target database and product rule.

Edge matrix

Edge	Required decision
bulk update bypasses callbacks	database/application audit alternative and tests
background task has no user principal	explicit system actor
clock differs across nodes	authoritative instant/source
deleted row referenced by child	restore and referential policy
global filters hide rows	administrative/reconciliation path with authorization

Quick check

1. When is soft delete justified, and when is it not enough?
2. What is the risk of no lifecycle-specific methods?
3. Why are auditable fields also a debugging tool?

Debugging exercise

A bulk report includes deleted rows and active rows because filter condition is missing.

How do you fix query layer and index layer?

Expected: lifecycle-aware repository method, explicit compound index on status+tenant, and regression tests.

Practice

- **Foundation:** Add three lifecycle states to one domain entity.
- **Interview Core:** Explain soft delete with recovery and evidence requirements.
- **SDE-2 Follow-up:** Describe how you prevent accidental exposure of soft-deleted rows.

Interviewer question and model answer

Interviewer: Does `@CreatedDate` give me a compliance audit log?

Model answer: It gives convenient entity metadata when Spring Data lifecycle hooks run. It is not automatically immutable, complete, independently retained, or tamper-evident, and bulk/native writes can bypass it. I separate operational timestamps from a required audit/event record, define the actor and clock source, protect retention and access, and test every write path that claims to produce audit evidence.

SDE-2 CORE CHAPTER

Chapter 13 - Spring Data MongoDB Foundations

Spring Data MongoDB is useful when document modeling is the right fit, but document stores require explicit modeling trade-offs.

The mental shift

In SQL, joins are normalized by default.

In MongoDB, many relationships are denormalized by design.

The real question is not "SQL or NoSQL?" It is whether one bounded document can own the data and serve the important access patterns without unbounded growth or frequent cross-document invariants.

Core decisions for interviews

1. **Embedded vs referenced documents:** embed for read-side locality, reference when independent lifecycle is required.
2. **Indexing fields:** every filter and sort field should be indexed with access pattern in mind.
3. **Update model:** use atomic field updates over full-document writes when possible.

Repository mapping notes

Document repositories are familiar, but behavior differs:

- a single-document write is atomic; multi-document transactions are available on supported replica-set or sharded deployments but add coordination and do not repair a poor document boundary,
- large arrays and in-place updates need careful `$push/$set` usage,
- ordering of projections in large arrays is storage-dependent behavior.

Start with the native operation, then choose the abstraction:

JAVASCRIPT

```
db.orders.updateOne(
  { _id: orderId, version: expectedVersion },
  { $set: { status: "PAID" }, $inc: { version: 1 } }
)
```

`MongoTemplate` can express explicit update operators and inspect the matched/modified counts. A repository `save` of an entire document is not always the correct substitute for an atomic field update.

Failure matrix

Failure	Evidence/response
matched count is zero	missing document or version conflict; distinguish deliberately
duplicate key	classify the known unique index
transient transaction label	retry the whole transaction under MongoDB driver guidance and deadline
document/array grows without bound	redesign boundary or bucket/reference data
secondary read is stale	state read preference/read concern and client-visible guarantee

Interview-ready mini model

TEXT

```
Order document
   |
   +-- items[] (embedded)
   +-- totals (immutable-ish)
   +-- status audit timeline (subdocument)
```

If every query needs `items` and `statusHistory`, evaluate size and write amplification.

Quick check

1. Why can a document model simplify read APIs?
2. What is one risk of deeply embedded arrays?
3. How does Mongo transaction behavior differ from relational transaction assumptions?

Debugging exercise

Read endpoint gets slow after adding a nested array field.

List three first checks before redesigning the schema.

Expected:

- index coverage,
- document shape growth,
- projection and projection-only query path.

Practice

- **Foundation:** Draw one embedded and one referenced document alternative.
- **Interview Core:** Choose one model for order lines with read-heavy dashboards.
- **SDE-2 Follow-up:** Explain safe concurrency strategy for partially updated arrays.

Interviewer question and model answer

Interviewer: MongoDB supports transactions, so why care about embedding?

Model answer: Transactions make some multi-document changes possible on supported deployments, but they add coordination, latency, and failure handling. I still model the common invariant and read path inside one bounded document when that lifecycle fits. I reference independent or unbounded data. Then I define indexes, document growth, read/write concerns, and whether an atomic update with a version predicate is clearer than loading and saving the whole document.

SDE-2 CORE CHAPTER

Chapter 14 - Spring Data Redis for Cache and Rate-State

Redis is often used for caches, locks, and short-lived state. Spring Data Redis can help, but cache logic is not repository CRUD.

Cache decisions before repositories

1. Define TTL and eviction policy.
2. Define key namespace and collision prevention.
3. Define read/write strategy (write-through, write-behind, refresh-ahead).

Treat Redis as a separate data system with explicit persistence, replication, failover, and memory contracts. In this book it is a derived cache/coordination boundary; the dedicated Redis book covers system-of-record designs.

Repository integration pattern

TEXT

```
Primary DB transaction commits source state + version/outbox
        -> cache invalidation/update is retried idempotently
        -> read miss loads source and fills only if not older
```

There is no universal "always delete before or after the database write" recipe. Draw the race. A versioned key/value or source-version check can prevent an older fill from overwriting newer data. TTL limits staleness duration; it does not make stale data impossible.

Before `RedisTemplate` or Spring Cache, state the native atomic operation. For a fixed-window rate counter, `INCR` plus first-write expiry needs an atomic script/transactional design so a crash does not leave a counter without TTL. For cache-aside, state exact key, serialization version, source version, TTL jitter, timeout, and fallback.

TRY IT | Common interview questions

- When should TTL be short vs long?
- What happens during stampede after TTL expiration?
- How do you avoid cache inconsistency after writes?

WATCH OUT | Common mistakes

- Using repository-like methods for long-lived mutable counters.
- Caching every repository method result without measuring read/write ratio.
- Forgetting namespace versioning when schema changes.
- Treating `@Cacheable` as a stampede, consistency, or distributed-lock solution.
- Retrying Redis indefinitely while the source database is already overloaded.

Failure matrix

Failure	Required behavior
Redis timeout	bounded fallback or explicit failure by product contract
mass expiry	jitter, request coalescing, warmup, and source admission
stale fill after invalidation	source version/fencing or delayed-delete protocol
eviction before TTL	treat miss as normal; never infer source deletion
failover loses acknowledged cache write	source remains authoritative; repair/refill
distributed lock holder pauses	fencing token at protected resource, not lease alone

Practical checklist

- Key shape should include version and tenant.
- Use atomic ops for counters and rate checks.
- Provide fallback path when Redis is unavailable.

Quick check

1. Why is cache-aside easy and dangerous?
2. What is a stampede and how do you handle it?
3. Why do key naming contracts matter across teams?

Debugging exercise

A count endpoint using Redis returns stale values after a hot deploy.

What checks do you run?

Expected: check TTL reset rules, versioned key migration, and fallback read-through path.

Practice

- **Foundation:** Draft Redis key naming for one endpoint.
- **Interview Core:** Explain a safe fallback strategy for temporary Redis outage.
- **SDE-2 Follow-up:** Compare lock semantics in distributed counters with optimistic locking in DB writes.

Interviewer question and model answer

Interviewer: Can I add `@Cacheable` to every repository read and invalidate on writes?

Model answer: No. I first identify expensive stable reads, the source of truth, acceptable staleness, key and serialization version, TTL/eviction, tenant scope, and outage behavior. Annotation-based caching does not solve concurrent misses, stale fills, cross-instance invalidation, or hot keys. I test the write/read race, use version-aware invalidation where needed, bound Redis and source concurrency, and keep correctness independent of the cache unless the product explicitly chooses otherwise.

SDE-2 CORE CHAPTER

Chapter 15 - Testing Data Paths, Observability, and Regression Harness

Spring Data quality is proven through test selection, query diagnostics, and failure evidence.

Test layers

1. **Unit-level contract tests:** method behavior and parameter validation.
2. **Repository behavior tests:** query generation and result mapping rules.
3. **Integration tests with real store:** transaction, index, ordering, and concurrency behavior.
4. **Failure-path tests:** duplicates, stale versions, timeout and rollback.

The SD06 executable lab demonstrates these layers with Spring Boot, Spring Data JPA, Hibernate, and H2. H2 proves repository wiring and generic JPA contracts; it does **not** prove MySQL plans, gap locks, collation, or deadlock behavior. Those claims require the target engine, preferably through Testcontainers in the MySQL/Hibernate volumes.

Query observability model

```text
Repository call -> SQL / store statement log -> row count/latency -> result mapping -> business assertion
```

Interviews value evidence: capture query count and duration for critical operations.

Deterministic fixtures

Fixtures should be:

- ordered,
- repeatable,
- minimal,
- scoped to one behavior.

Avoid random fixtures that make flaky transaction tests pass/fail.

Claims and the test that can prove them

Claim	Minimum useful evidence
derived query order is deterministic	equal primary sort keys plus asserted ID tie-breaker
`Slice` continuation works	`size + 1` data, first/next content, `hasNext`
rollback protects state	invoke real service proxy and read in a fresh transaction
optimistic conflict is detected	two snapshots/transactions and stale version write
pessimistic mode is requested	active transaction and managed entity lock-mode assertion; target DB test for blocking
N+1 is absent	SQL statement count and bounded relationship data
query is fast	target-engine plan plus representative distribution/load

Observability without leaking data

Capture normalized operation name, duration, rows, query count, pool wait, lock wait, conflict/retry count, and result size. Do not tag metrics with raw SQL, customer ID, request key, or arbitrary repository method parameters. Logs may carry controlled correlation and a sanitized query fingerprint.

Quick check

1. Which test layer catches N+1 first?
2. Why do in-memory databases hide query shape?
3. What evidence proves a lock path is actually exercised?

Debugging exercise

Repository integration tests pass locally but fail against target store.

List investigation steps.

Expected: check driver/version compatibility, timezone/ordering, transaction isolation defaults, and index availability.

Practice

- **Foundation:** Write a repository test for stable ordering and page shape.
- **Interview Core:** Add assertions for no-over-fetch and bounded response size.
- **SDE-2 Follow-up:** Create a failure test for optimistic lock exception recovery.

Interviewer question and model answer

Interviewer: Why is `@DataJpaTest` with H2 not enough?

Model answer: It is excellent for repository wiring, JPQL parsing, mapping, and many lifecycle contracts. It cannot prove the target database's dialect, collation, indexes, optimizer plan, isolation anomalies, lock ranges, deadlocks, or online migration behavior. I keep fast H2 tests for feedback and add a smaller set of Testcontainers tests against the production engine for every claim that depends on that engine.

SDE-2 CORE CHAPTER

Chapter 16 - Common Interview Traps for Spring Data

Interviewers rarely ask for one repository method name. They ask whether behavior is safe under real constraints.

Top traps

1. Assuming `findAll()` is harmless for large data.
2. Forgetting deterministic sort.
3. Ignoring repository method side effects in transactions.
4. Using broad delete methods.
5. Missing pagination or cursor semantics.
6. Treating optimistic lock exceptions as transient and never handling retries.
7. Believing native queries are automatically faster.
8. Assuming soft-delete means secure-by-default.
9. Assuming `getBy...` universally throws when no row exists.
10. Counting when the requirement is only existence.
11. Mixing a cursor with a numeric offset.
12. Treating `REQUIRES_NEW` as retry.
13. Retrying a stale entity without re-reading/reapplying the command.
14. Assuming MongoDB has no transactions or that transactions remove modeling costs.
15. Claiming lock escalation is a portable database/JPA behavior.

Code-read trap

When you read one long repository method name, inspect:

- null checks,
- sorting, including tie-breakers,
- return type,
- checked exceptions in callers,
- surrounding transaction boundaries.

Fast interview answer framework

For each trap, answer:

- What is the hidden behavior?
- Why can it fail in production?
- What one safer design change solves it?

Debugging exercise

Method: `Page<User> findAllByCreatedAtAfterOrderByCreatedAtDesc(Date from, Pageable page);`

Observed duplicate and skipped rows while clients traverse offset pages during inserts.

What do you fix, and what can no live pagination method promise?

Expected: add a stable secondary order (`OrderByCreatedAtDescIdDesc`), align an index to the filter/order, and use a keyset cursor when deep/live traversal semantics fit. Explain that live cursor traversal is not a snapshot when sort keys mutate; an exhaustive export may need snapshot/materialization semantics.

Practice

- **Foundation:** Pick two traps and map cause-and-fix.
- **Interview Core:** Turn one trap into one precise interview answer.
- **SDE-2 Follow-up:** Explain when repository convenience hides isolation-level risk.

Interviewer question and model answer

Interviewer: A repository method is one line and passes unit tests. What evidence do you still request before approving a hot endpoint?

Model answer: I request the generated SQL or native command, parameters by shape, projection width, deterministic order, result bound, query/count statement count, target-engine plan with representative distribution, transaction and lock duration, pool demand, and failure behavior for missing, duplicate, timeout, and concurrent updates. Then I add a regression test for the specific contract. Repository brevity is not performance or correctness evidence.

SDE-2 CORE CHAPTER

Chapter 17 - Practice, Solutions, Readiness, and Sources

The final section is your bridge from reading to interviews.

Cumulative assessment 1 - Design discipline

1. Design three repository methods for one aggregate.
2. Define one boundary where repository abstraction must stop.
3. Add optimistic locking and one conflict recovery branch.
4. Write ordering strategy for pagination.
5. Explain failure behavior for duplicate IDs.

Solution sketch

Use narrow interfaces, explicit state transitions, deterministic sort keys, and explicit exception handling for every command path. Add idempotent retries only around safe state transitions and track user-visible error contracts.

Cumulative assessment 2 - Correctness and performance

1. Show one derived method that is too broad.
2. Convert it to declared query and projection.
3. Identify potential N+1 risks.
4. Add query-count assertion in a test.
5. Add one lock strategy for conflicting writes.

Solution sketch

Use explicit select + join intent in the declared query, remove unnecessary eager graph, and add deterministic paging. Add assertions for page window size and ordering.

Cumulative assessment 3 - Persistence store boundaries

1. Choose repository strategy for one SQL feature.
2. Choose repository strategy for one document feature.
3. Choose Redis strategy for one coordination task.
4. Define rollback and cache handling on write failure.
5. Explain observability evidence required for each.

Solution sketch

Choose SQL when relational constraints, joins, and multi-row invariants fit the model. Choose MongoDB when a bounded document owns the lifecycle and serves the access pattern; acknowledge supported multi-document transactions without using them to excuse a poor boundary. Use Redis here for derived TTL/rate/cache state with an explicit source, atomic operation, expiry, failover, and fallback contract. Observe query/command shape, result size, latency, lock/wait, retry, and stale-data evidence.

Cumulative assessment 4 - Mock interview round

Given this API description, explain repository choices and failure handling:

- Create order
- Pay order
- Cancel order
- List open orders with pagination

Solution sketch

Model command-oriented repositories, ensure idempotent order creation, isolate payment side-effects from transaction-critical DB updates, and paginate `OPEN` orders deterministically.

Cumulative assessment 5 - Readiness check

1. Do you explain repository behavior without naming only annotations?
2. Can you describe where abstraction hides cost?
3. Can you propose one test for lock contention?
4. Can you explain a deterministic paging strategy?
5. Can you map one bug directly to a query/call trace?

Solution sketch

Readiness is demonstrated when you can discuss contract, semantics, failure handling, and evidence in one coherent flow.

Predict the outcome

1. Derived query for bounded page without `OrderBy` can still be paginated but nondeterministic.
2. Optimistic lock retry without refresh can still rethrow repeatedly.
3. Mongo schema changes may require migration scripts and read-rewrite strategy.
4. Redis count drift usually appears when cache and DB update ordering are inconsistent.

Twenty solved interviewer follow-ups

Read the question, answer aloud, then compare your answer with the model. A strong response names contract, runtime work, failure, and proof.

1. When would you avoid a repository for one operation?

Model answer: I use a custom fragment, `EntityManager`, template, or JDBC when the operation needs vendor SQL, bulk mutation, window functions, a deterministic lock sequence, or a narrow read model whose cost should remain visible. I keep repositories for the rest of the aggregate; the choice is per operation.

2. What happens when save receives a new versus detached JPA entity?

Model answer: Spring Data JPA selects persistence behavior from entity-new detection. A new entity is generally persisted; an existing/detached entity is generally merged, and merge copies state into a managed instance rather than reattaching the argument. I avoid binding an HTTP entity graph and blindly saving it; I load authoritative state, authorize, apply a command, and flush inside the service transaction.

3. Does getByEmail throw when no row exists?

Model answer: The `get` prefix alone does not define that guarantee. I inspect the declared return type, nullability rules, module/version, and cardinality. I prefer `Optional` for normal absence or an application-owned `require...` method that translates absence deliberately. `getReferenceById` is a separate JPA lazy-reference API.

4. Why use existsBy... instead of countBy... > 0?

Model answer: Existence is the actual question and can stop after one match; count must calculate the number of matches. I still inspect generated SQL and indexes. Neither is a business-table health check.

5. Page or Slice?

Model answer: I use `Page` only when the client truly needs an exact total because it can add a count query. `Slice` communicates content plus continuation without promising a total. For deep live traversal I consider a keyset `Window`/cursor with a total order.

6. Why must a cursor include a unique tie-breaker?

Model answer: If several rows share `createdAt`, a cursor containing only that timestamp cannot identify the exact continuation boundary, so rows can be skipped or repeated. I order by `(createdAt,id)` and carry both values with matching direction, scope, and filters.

7. Can I combine cursor and offset pagination?

Model answer: Not as one continuation contract. Offset advances by position; a cursor advances from an ordered key tuple. Combining them obscures semantics and can reintroduce drift. I choose one, cap the window, and document live versus snapshot behavior.

8. How do you diagnose N+1?

Model answer: I reproduce a bounded endpoint, capture SQL statement count and rows, identify which association access triggers secondary selects, and compare fetch join/entity graph, batch fetch, and DTO projection. The regression test asserts query count and result shape; a target-engine test covers the plan.

9. Why not mark every relationship eager?

Model answer: Static eagerness loads relationships for use cases that do not need them and can create large joins/object graphs. I choose a per-query fetch plan and keep to-many paging bounded. One SQL statement is not automatically cheap if it returns a Cartesian row explosion.

10. What exactly does flush guarantee?

Model answer: Flush synchronizes pending persistence-context changes to the database transaction so SQL and constraints can run. It does not commit, release every lock, make a remote call atomic, or prove another transaction can see the change. A failure means the transaction should roll back.

11. Is REQUIRES_NEW a retry strategy?

Model answer: No. It creates an independent physical transaction while suspending the caller's context and may require another connection. Retry starts after a classified failed transaction rolls back; it re-reads state, reapplies a safe command, preserves logical identity, and obeys attempt/deadline limits.

12. How do you handle an optimistic conflict?

Model answer: I return a conflict when user reconciliation is required, or I perform a bounded retry only if the command is still valid and idempotent. Each attempt uses a fresh transaction and refreshed state. I never retry every exception or repeatedly save the same stale object.

13. When is pessimistic locking justified?

Model answer: It can fit a short high-contention critical section where bounded waiting is preferable to frequent optimistic failures. I need an active transaction, an index-supported predicate, timeout policy, consistent acquisition order, and target-database tests. JPA lock modes do not erase vendor behavior.

14. Are database locks guaranteed to escalate?

Model answer: That is vendor-specific, not a Spring Data/JPA guarantee. I name the engine, version, isolation, statement, index, and documented row/key/range/predicate behavior, then inspect lock-wait or deadlock evidence rather than repeating a universal escalation rule.

15. When do you choose a native query?

Model answer: When a measured important path needs a database feature or SQL shape that JPQL cannot express clearly. I bind values, keep projection and ordering explicit, supply a correct count only if needed, pin target-engine tests, and record the portability/migration cost.

16. What can go wrong with a bulk update?

Model answer: Bulk JPQL/native DML bypasses normal per-entity dirty checking and callbacks, so managed objects can become stale. I define affected-row expectations, auditing/version behavior, clear or refresh the persistence context, and test concurrent and rollback cases.

17. Does MongoDB's transaction support make document modeling irrelevant?

Model answer: No. Supported deployments provide multi-document transactions, but they add coordination and failure handling. I still choose bounded document ownership from access patterns and invariants, then state read/write concerns, indexes, growth, and atomic update/version behavior.

18. Does @Cacheable solve cache stampedes?

Model answer: Not by itself. I define source of truth, key/version, TTL jitter, concurrent-miss coalescing, source admission, stale-fill prevention, and Redis outage behavior. Correctness must not silently depend on the cache unless that is an explicit product contract.

19. Why can an H2 repository test pass while MySQL fails?

Model answer: Dialect syntax, collation, temporal conversion, indexes, plans, isolation, lock ranges, deadlocks, and constraints can differ. H2 remains useful for fast mapping/query feedback; every engine-specific claim gets a target MySQL Testcontainers test with representative schema and data.

20. What evidence do you present for a repository design?

Model answer: I show the application contract, generated/native query, bound result and deterministic order, projection width, query/count count, target plan, transaction and lock boundary, pool demand, failure matrix, and a regression test. That is stronger than saying an annotation is convenient or that a method is `O(1)`.

Cross-book boundaries

- Transaction fundamentals and AOP semantics -> **SD 04 - Spring Framework**.
- Auto-configuration and app packaging -> **SD 05 - Spring Boot**.
- ORM mapping and JPA behavior -> **SD 03 - Hibernate and JPA**.
- Query and indexing fundamentals -> **SD 02 - MySQL**.
- Cache and cache patterns -> **SD 08 - Redis**.
- Distributed architecture trade-offs -> **SD 09 - Apache Kafka and Spring Kafka**.

Primary sources

- Spring Data Commons Reference: `https://docs.spring.io/spring-data/commons/reference/`
- Spring Data JPA Reference: `https://docs.spring.io/spring-data/jpa/reference/`
- Spring Data MongoDB Reference: `https://docs.spring.io/spring-data/mongodb/reference/`
- Spring Data Redis Reference: `https://docs.spring.io/spring-data/redis/reference/`
- Spring Framework transaction model: `https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative.html`
- Java concurrency and locking model: `https://docs.oracle.com/en/java/javase/21/docs/`

SDE-2 CORE CHAPTER

Chapter 18 - Java 21 Spring Data Readiness Companion

This dependency-free executable model validates repository contracts, deterministic query intent, pagination boundary checks, optimistic concurrency responses, store-boundary decisions, and recovery policy for Spring Data-centric interview scenarios.

JAVA (continued 1/12)

```java
import java.time.Duration;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.Queue;
import java.util.Set;

/**
 * Dependency-free Java 21 companion for Spring Data interview reasoning.
 *
 * <p>
 * The model keeps Spring Data discussion precise:
 * repository contracts, deterministic pagination, fetch strategy,
 * transactional intent, and consistency recovery semantics.
 */
public final class SpringDataInterviewCompanion {
    private SpringDataInterviewCompanion() {
    }

    public static void main(String[] args) {
        validatesMethodContract();
        validatesDeterministicOrdering();
        validatesPaginationModels();
        validatesLockRetryPolicy();
        validatesStoreBoundaryChoice();
        validatesObservabilitySignals();
        System.out.println("SpringDataInterviewCompanion checks passed");
```

JAVA (continued 2/12)

```
    }

    private static void validatesMethodContract() {
        RepositoryContract create = new RepositoryContract(
                "save", "Order", true, false, true, false);
        RepositoryContract find = new RepositoryContract(
                "findByStatusOrderByCreatedAtDescIdDesc", "Order",
                true, true, false, true);
        RepositoryContract delete = new RepositoryContract(
                "deleteById", "Order", false, false, true, false);

        require(create.pureCommand(), "save must be a command-oriented method");
        require(find.declaresOrdering(), "query name must declare its ordering intent");
        require(!delete.implicitFilter(), "deleteById is scoped by explicit identity filter");
        require(find.pagingSafe(), "querying latest style should be paging-safe");
    }

    private static void validatesDeterministicOrdering() {
        List<RecordRow> rows = List.of(
                new RecordRow(7L, "OPEN", 100),
                new RecordRow(5L, "OPEN", 100),
                new RecordRow(9L, "OPEN", 101));

        List<RecordRow> byCreatedAtDesc = rows.stream()
                .sorted(Comparator.comparingInt(RecordRow::createdAt).reversed())
                .toList();

        List<RecordRow> withTieBreaker = new ArrayList<>(rows);
        Comparator<RecordRow> newestFirst = Comparator
                .comparingInt(RecordRow::createdAt)
                .reversed()
                .thenComparing(RecordRow::id, Comparator.reverseOrder());
        withTieBreaker.sort(newestFirst);
```

JAVA (continued 3/12)

```
        require(
                byCreatedAtDesc.getFirst().createdAt() == 101,
                "createdAt order still must place largest first");
        require(withTieBreaker.get(0).id() == 9L,
                "primary descending timestamp must remain primary");
        require(withTieBreaker.get(1).id() == 7L,
                "descending id must break equal-timestamp ties");
    }

    private static void validatesPaginationModels() {
        OffsetPage firstOffsetPage = new OffsetPage(0, 20);
        OffsetPage secondOffsetPage = firstOffsetPage.next();
        require(secondOffsetPage.offset() == 20,
                "offset pagination advances by page size");

        CursorPage firstCursorPage = CursorPage.first(20);
        CursorPage nextCursorPage = firstCursorPage.after(100, 3L);
        require(nextCursorPage.afterCreatedAt().orElseThrow() == 100,
                "cursor retains the ordered timestamp");
        require(nextCursorPage.afterId().orElseThrow() == 3L,
                "cursor retains the unique tie-breaker");

        require(!OffsetPage.unbounded().pagingSafe(),
                "unbounded offset work is not safe");
        require(!CursorPage.first(Integer.MAX_VALUE).pagingSafe(),
                "unbounded cursor work is not safe");
    }

    private static void validatesLockRetryPolicy() {
        OptimisticWriteAttempt first = OptimisticWriteAttempt.initial();
        OptimisticWriteAttempt second = first.afterFailure(
                new StaleWriteConflict("version mismatch"));
        OptimisticWriteAttempt third = second.afterFailure(
                new StaleWriteConflict("version mismatch"));
```

JAVA (continued 4/12)

```
        OptimisticWriteAttempt fourth = third.afterFailure(
                new StaleWriteConflict("version mismatch"));
        OptimisticWriteAttempt permanent = first.afterFailure(
                new PermanentStoreFailure("invalid query"));

        require(
                second.outcome() == AttemptOutcome.RETRYING,
                "first conflict should require refresh and retry");
        require(
                third.outcome() == AttemptOutcome.RETRYING,
                "second conflict should also require retry policy");
        require(
                fourth.outcome() == AttemptOutcome.FAIL_FAST,
                "too many retries should fail fast");
        require(permanent.outcome() == AttemptOutcome.FAIL_FAST,
                "unclassified failures must never be retried blindly");
    }

    private static void validatesStoreBoundaryChoice() {
        StoreDecision sql = chooseStore(new Workload(
                true, true, false, false, false));
        StoreDecision mongo = chooseStore(new Workload(
                true, false, true, true, false));
        StoreDecision redis = chooseStore(new Workload(
                false, false, false, false, true));

        require(sql.choice() == StoreChoice.SQL,
                "relational joins and multi-row invariants point to SQL");
        require(mongo.choice() == StoreChoice.MONGODB,
                "bounded document locality can point to MongoDB");
        require(mongo.caveat().contains("transactions"),
                "MongoDB transaction capability must not be denied categorically");
        require(redis.choice() == StoreChoice.REDIS,
```

JAVA (continued 5/12)

```
                "derived expiring state can point to Redis");
    }

    private static void validatesObservabilitySignals() {
        var signals = Map.of(
                "queryCount", "4",
                "p50LatencyMs", "45",
                "lockWaitMs", "12",
                "retryCount", "1");

        require(!signals.containsKey("error"), "successful path must not emit error markers");
        require(
                Integer.parseInt(signals.get("queryCount")) <= 6,
                "query count should stay bounded for endpoint contract");

        DiagnosticTimeline timeline = new DiagnosticTimeline(signals);
        List<String> keys = timeline.sortedKeys();
        require(
                keys.get(0).equals("lockWaitMs"),
                "sorted signal output keeps deterministic trace order");
        require(timeline.p99LatencyUs() >= 0, "latency math should be non-negative");
    }

    private static void require(boolean condition, String message) {
        if (!condition) {
            throw new AssertionError(message);
        }
    }

    record RepositoryContract(
            String method,
            String aggregate,
            boolean returnsEntity,
```

JAVA (continued 6/12)

```
            boolean mayReturnMultiple,
            boolean mutates,
            boolean hasPagingIntent) {

        boolean pureCommand() {
            return mutates && returnsEntity && !mayReturnMultiple;
        }

        boolean declaresOrdering() {
            String normalized = method.toLowerCase(Locale.ROOT);
            return normalized.contains("find") && normalized.contains("orderby");
        }

        boolean implicitFilter() {
            return normalizedName().contains("by")
                    && !method.contains("All")
                    && mayReturnMultiple;
        }

        boolean pagingSafe() {
            if (!hasPagingIntent) {
                return false;
            }
            return normalizedName().contains("orderbycreatedatdesc")
                    && normalizedName().contains("id");
        }

        private String normalizedName() {
            return method.toLowerCase(Locale.ROOT);
        }
    }

    record RecordRow(long id, String status, int createdAt) {
```

JAVA (continued 7/12)

```
    }

    record OffsetPage(int offset, int size) {
        OffsetPage {
            if (offset < 0) {
                throw new IllegalArgumentException("offset must be nonnegative");
            }
            if (size <= 0) {
                throw new IllegalArgumentException("size must be positive");
            }
        }

        static OffsetPage unbounded() {
            return new OffsetPage(0, Integer.MAX_VALUE);
        }

        OffsetPage next() {
            return new OffsetPage(Math.addExact(offset, size), size);
        }

        boolean pagingSafe() {
            return size < Integer.MAX_VALUE;
        }
    }

    record CursorPage(int size, Optional<Integer> afterCreatedAt,
                      Optional<Long> afterId) {
        CursorPage {
            if (size <= 0) {
                throw new IllegalArgumentException("size must be positive");
            }
            Objects.requireNonNull(afterCreatedAt, "afterCreatedAt");
            Objects.requireNonNull(afterId, "afterId");
```

JAVA (continued 8/12)

```
            if (afterCreatedAt.isPresent() != afterId.isPresent()) {
                throw new IllegalArgumentException(
                        "cursor timestamp and tie-breaker must appear together");
            }
        }

        static CursorPage first(int size) {
            return new CursorPage(size, Optional.empty(), Optional.empty());
        }

        CursorPage after(int createdAt, long id) {
            return new CursorPage(size, Optional.of(createdAt), Optional.of(id));
        }

        boolean pagingSafe() {
            return size < Integer.MAX_VALUE;
        }
    }

    enum AttemptOutcome {
        SUCCESS,
        RETRYING,
        FAIL_FAST
    }

    static final class StaleWriteConflict extends RuntimeException {
        private static final long serialVersionUID = 1L;

        StaleWriteConflict(String message) {
            super(message);
        }
    }
```

JAVA (continued 9/12)

```
    static final class PermanentStoreFailure extends RuntimeException {
        private static final long serialVersionUID = 1L;

        PermanentStoreFailure(String message) {
            super(message);
        }
    }

    static final class OptimisticWriteAttempt {
        private final int attempts;
        private final AttemptOutcome outcome;

        OptimisticWriteAttempt(int attempts, AttemptOutcome outcome) {
            this.attempts = attempts;
            this.outcome = outcome;
        }

        static OptimisticWriteAttempt initial() {
            return new OptimisticWriteAttempt(0, AttemptOutcome.SUCCESS);
        }

        OptimisticWriteAttempt afterFailure(Throwable cause) {
            Objects.requireNonNull(cause, "cause");
            int failedAttempts = attempts + 1;
            if (!(cause instanceof StaleWriteConflict) || failedAttempts >= 3) {
                return new OptimisticWriteAttempt(failedAttempts,
                        AttemptOutcome.FAIL_FAST);
            }
            return new OptimisticWriteAttempt(failedAttempts,
                    AttemptOutcome.RETRYING);
        }

        AttemptOutcome outcome() {
```

JAVA (continued 10/12)

```
            return outcome;
        }
    }

    enum StoreChoice {
        SQL,
        MONGODB,
        REDIS,
        NEEDS_MORE_EVIDENCE
    }

    record Workload(boolean sourceOfTruth,
                    boolean relationalJoins,
                    boolean documentLocality,
                    boolean boundedDocument,
                    boolean expiringDerivedState) {
    }

    record StoreDecision(StoreChoice choice, String reason, String caveat) {
    }

    static StoreDecision chooseStore(Workload workload) {
        Objects.requireNonNull(workload, "workload");
        if (workload.sourceOfTruth() && workload.relationalJoins()) {
            return new StoreDecision(StoreChoice.SQL,
                    "relational constraints and join-shaped access",
                    "verify isolation, indexes, and engine behavior");
        }
        if (workload.sourceOfTruth() && workload.documentLocality()
                && workload.boundedDocument()) {
            return new StoreDecision(StoreChoice.MONGODB,
                    "bounded aggregate reads and document locality",
                    "multi-document transactions exist on supported deployments, "
```

JAVA (continued 11/12)

```
                        + "but frequent cross-document invariants may signal a poor model");
        }
        if (!workload.sourceOfTruth() && workload.expiringDerivedState()) {
            return new StoreDecision(StoreChoice.REDIS,
                    "derived state with an explicit expiration contract",
                    "define source of truth, eviction, failover, and stale-data policy");
        }
        return new StoreDecision(StoreChoice.NEEDS_MORE_EVIDENCE,
                "requirements do not select a store",
                "measure query shape, consistency, growth, and operations");
    }

    static final class DiagnosticTimeline {
        private final Map<String, String> values;

        DiagnosticTimeline(Map<String, String> values) {
            this.values = new LinkedHashMap<>(values);
        }

        List<String> sortedKeys() {
            return new ArrayList<>(values.keySet())
                    .stream()
                    .sorted()
                    .toList();
        }

        int p99LatencyUs() {
            return sortedKeys().stream()
                    .filter(name -> name.toLowerCase(Locale.ROOT).contains("latency"))
                    .findFirst()
                    .map(values::get)
                    .map(Integer::parseInt)
                    .map(ms -> ms * 1000)
```

JAVA (continued 12/12)

```
                    .orElse(0);
        }
    }

    // Minimal realistic examples used by interview explanation examples
    static final class Backoff {
        private final Queue<Duration> delays = new ArrayDeque<>();

        Backoff() {
            delays.add(Duration.ofMillis(10));
            delays.add(Duration.ofMillis(20));
            delays.add(Duration.ofMillis(30));
        }

        List<Long> drain() {
            List<Long> values = new ArrayList<>();
            while (!delays.isEmpty()) {
                values.add(delays.remove().toMillis());
            }
            return values;
        }

        boolean monotonic() {
            List<Long> values = drain();
            for (int i = 1; i < values.size(); i++) {
                if (values.get(i) < values.get(i - 1)) {
                    return false;
                }
            }
            return true;
        }
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: persistence boundaries, CRUD contracts, deterministic order, and query naming
- Interview Core: pagination, derived vs declared queries, locking, and fetch strategy
- SDE-2 Follow-up: MongoDB/Redis boundary design, transaction recovery, query optimization
- Readiness: complete 20 solved interview follow-up scenarios and five cumulative assessments

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: FW 05 - Spring Boot and REST APIs

Series index

Next book: FW 07 - MongoDB for Java Backend Interviews

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Language, OOP, and Modern Java
Java	**JAVA 05**	Collections, Streams, and I/O
Java	**JAVA 06**	JVM and Java Execution
Java	**JAVA 07**	Java Concurrency and the Memory Model
Java	**JAVA 08**	Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Java Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
Frameworks	**FW 01**	MySQL for Java Backend Interviews
Frameworks	**FW 02**	Hibernate and JPA for Java Backend Interviews
Frameworks	**FW 03**	Spring Framework for Java Engineers
Frameworks	**FW 04**	Spring Boot for Java Backend Engineers
Frameworks	**FW 05**	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	**FW 06**	Spring Data for Java Backend Engineers
Frameworks	**FW 07**	MongoDB for Java Backend Interviews
Frameworks	**FW 08**	Redis for Java Backend Interviews
Frameworks	**FW 09**	Persistence, SQL, JPA, and Caching
Frameworks	**FW 10**	Apache Kafka and Spring Kafka
Frameworks	**FW 11**	Spring Ecosystem Extensions
Frameworks	**FW 12**	Spring AI for Java Engineers
System Design	**SD 01**	Design, Backend, Testing, and Security
System Design	**SD 02**	Distributed Systems and System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Frameworks, Data, and Messaging - Book 06 of 12 - Study Step FW-06. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.